

EDS ACTIVITY NO:2 (IPL DATASET)

NAME: Parth Dinesh Nawarkar

PRN: 202501040365

Batch:CS4

Csv dataset of ipl

```
ipl.csv > data
1  am1,team2,winner,toss_winner,toss_decision,player_of_match,venue,runs,wickets
2  ans,Chennai Super Kings,Chennai Super Kings,Mumbai Indians,bat,Dwayne Bravo,Wankhede Stadium,165,5
3  engers Bangalore,Kolkata Knight Riders,Kolkata Knight Riders,Kolkata Knight Riders,field,Sunil Narine,Eden Gardens,176,4
4  als,Kings XI Punjab,Kings XI Punjab,Delhi Capitals,bat,KL Rahul,Delhi,180,6
5  ans,Delhi Capitals,Mumbai Indians,Delhi Capitals,field,Jasprit Bumrah,Mumbai,150,7
6  er Kings,Royal Challengers Bangalore,Chennai Super Kings,Chennai Super Kings,bat,MS Dhoni,Chennai,170,5
7  ght Riders,Kings XI Punjab,Kolkata Knight Riders,Kings XI Punjab,field,Andre Russell,Kolkata,190,3
8  ans,Kolkata Knight Riders,Mumbai Indians,Mumbai Indians,bat,Rohit Sharma,Dubai,200,2
9  als,Chennai Super Kings,Delhi Capitals,Chennai Super Kings,field,Shreyas Iyer,Sharjah,185,4
10 engers Bangalore,Kings XI Punjab,Kings XI Punjab,Royal Challengers Bangalore,bat,Mayank Agarwal,Dubai,210,2
11 per Kings,Kolkata Knight Riders,Chennai Super Kings,Kolkata Knight Riders,field,Faf du Plessis,Abu Dhabi,192,3
```

PROBLEM STATEMENT1

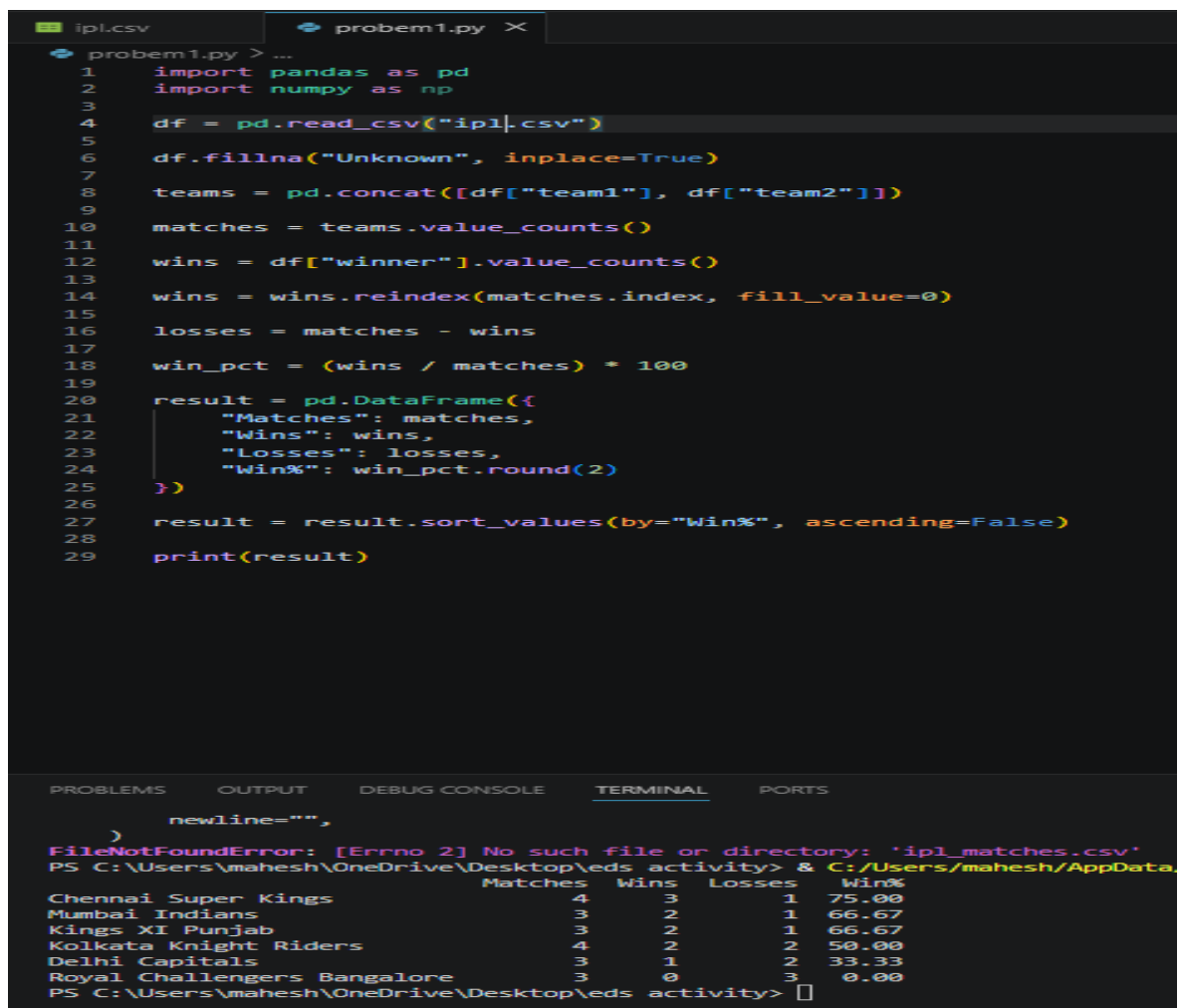
Calculate matches, wins, losses, win % for each team.

Input

team1, team2, winner

Output

Team stats table



The screenshot shows a code editor with a file named `problem1.py` open. The script reads a CSV file `ipl.csv`, processes the data to calculate matches, wins, losses, and win percentage for each team, and prints the results. The terminal output shows a `FileNotFoundError` for `ipl_matches.csv` and a table of team statistics.

```
problem1.py > ...
1 import pandas as pd
2 import numpy as np
3
4 df = pd.read_csv("ipl.csv")
5
6 df.fillna("Unknown", inplace=True)
7
8 teams = pd.concat([df["team1"], df["team2"]])
9
10 matches = teams.value_counts()
11
12 wins = df["winner"].value_counts()
13
14 wins = wins.reindex(matches.index, fill_value=0)
15
16 losses = matches - wins
17
18 win_pct = (wins / matches) * 100
19
20 result = pd.DataFrame({
21     "Matches": matches,
22     "Wins": wins,
23     "Losses": losses,
24     "Win%": win_pct.round(2)
25 })
26
27 result = result.sort_values(by="Win%", ascending=False)
28
29 print(result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

    )
    newline="",
FileNotFoundError: [Errno 2] No such file or directory: 'ipl_matches.csv'
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> & C:/Users/mahesh/AppData.
    Matches Wins Losses Win%
Chennai Super Kings      4      3      1  75.00
Mumbai Indians           3      2      1  66.67
Kings XI Punjab          3      2      1  66.67
Kolkata Knight Riders     4      2      2  50.00
Delhi Capitals            3      1      2  33.33
Royal Challengers Bangalore 3      0      3   0.00
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> 
```

PROBLEM STATEMENT2

Check whether the team winning the toss also wins the match and calculate the success percentage.

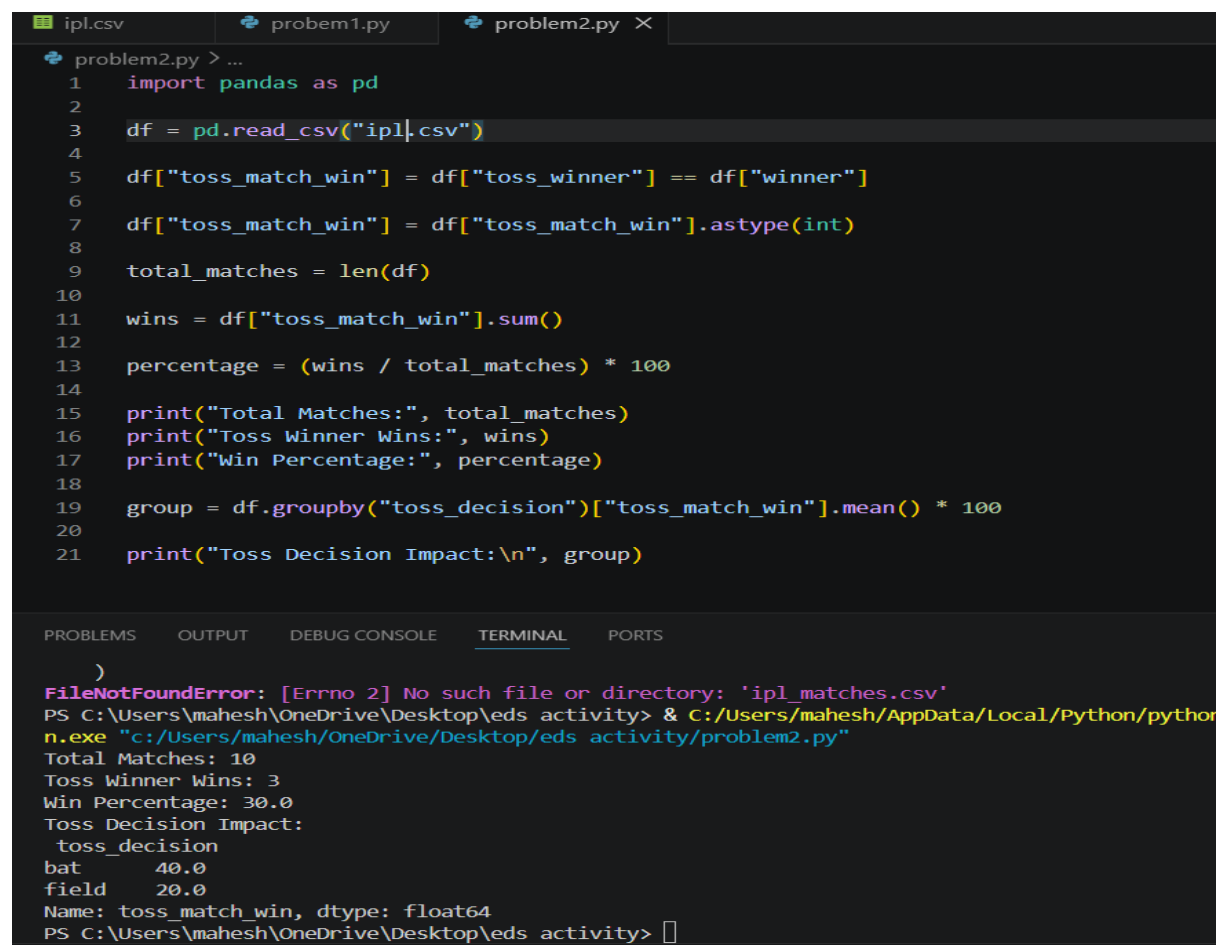
Input

CSV file: ipl.csv

Columns used: toss_winner, winner, toss_decision

Output

- Total matches
- Toss winner wins
- Win percentage
- Toss decision wise success



```
ipl.csv  problem1.py  problem2.py X
problem2.py > ...
1  import pandas as pd
2
3  df = pd.read_csv("ipl.csv")
4
5  df["toss_match_win"] = df["toss_winner"] == df["winner"]
6
7  df["toss_match_win"] = df["toss_match_win"].astype(int)
8
9  total_matches = len(df)
10
11 wins = df["toss_match_win"].sum()
12
13 percentage = (wins / total_matches) * 100
14
15 print("Total Matches:", total_matches)
16 print("Toss Winner Wins:", wins)
17 print("Win Percentage:", percentage)
18
19 group = df.groupby("toss_decision")["toss_match_win"].mean() * 100
20
21 print("Toss Decision Impact:\n", group)

)
FileNotFoundError: [Errno 2] No such file or directory: 'ipl_matches.csv'
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> & C:/Users/mahesh/AppData/Local/Python/python
n.exe "c:/Users/mahesh/OneDrive/Desktop/eds activity/problem2.py"
Total Matches: 10
Toss Winner Wins: 3
Win Percentage: 30.0
Toss Decision Impact:
  toss_decision
bat      40.0
field    20.0
Name: toss_match_win, dtype: float64
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> 
```

PROBLEM STATEMENT3

Find top players with most Player of Match awards.

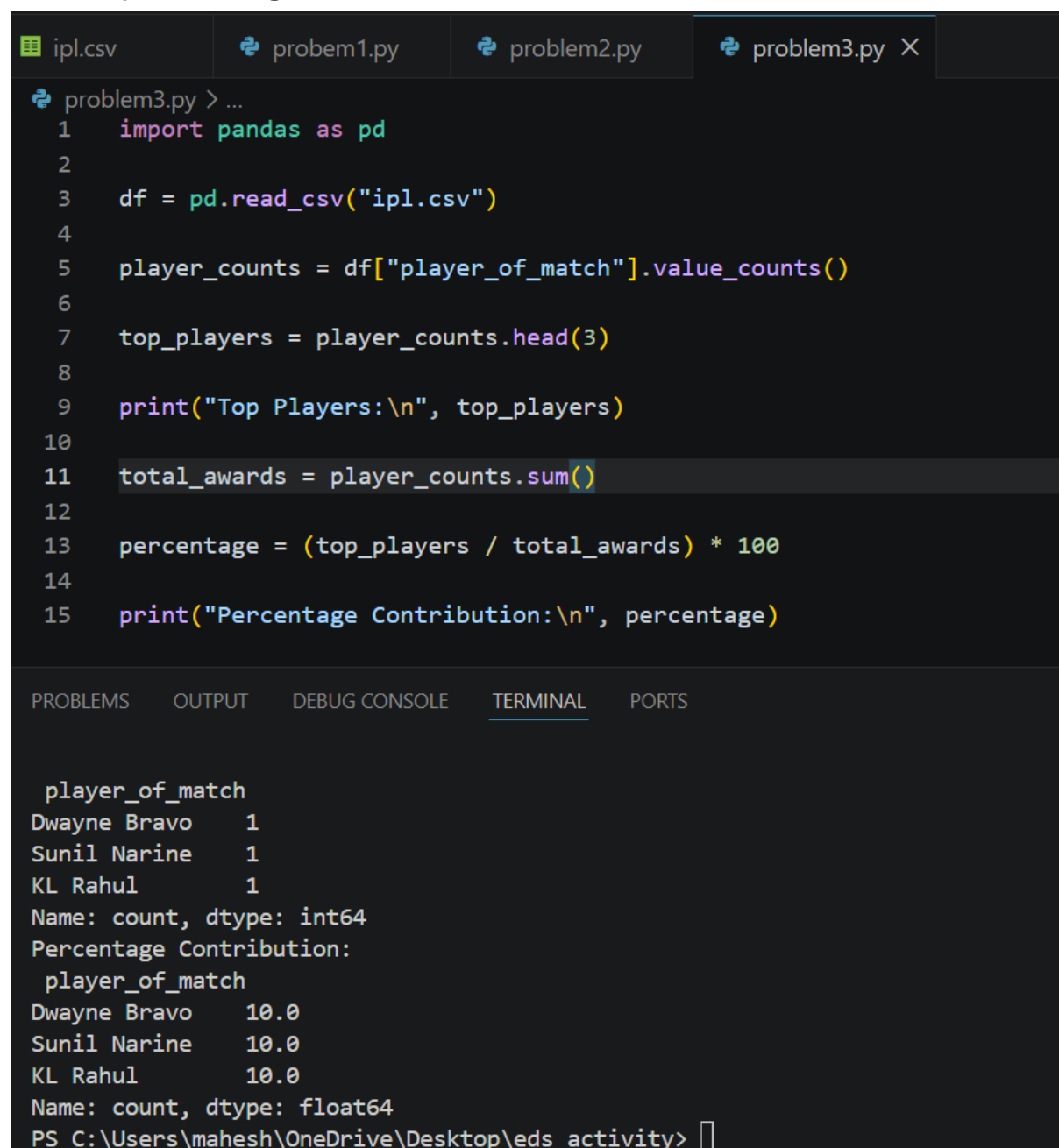
Input

Column: player_of_match

Output

Top players

Award percentage



```
ipl.csv  problem1.py  problem2.py  problem3.py X

problem3.py > ...
1  import pandas as pd
2
3  df = pd.read_csv("ipl.csv")
4
5  player_counts = df["player_of_match"].value_counts()
6
7  top_players = player_counts.head(3)
8
9  print("Top Players:\n", top_players)
10
11 total_awards = player_counts.sum()
12
13 percentage = (top_players / total_awards) * 100
14
15 print("Percentage Contribution:\n", percentage)

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

player_of_match
Dwayne Bravo    1
Sunil Narine    1
KL Rahul        1
Name: count, dtype: int64
Percentage Contribution:
player_of_match
Dwayne Bravo    10.0
Sunil Narine    10.0
KL Rahul        10.0
Name: count, dtype: float64
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> 
```

PROBLEM STATEMENT4

Find which team wins most matches at each venue.

Input

Columns: venue, winner

Output

Top team per venue

```
ipl.csv  problem1.py  problem2.py  problem3.py  problem4.py X
problem4.py > ...
1  import pandas as pd
2
3  df = pd.read_csv("ipl.csv")
4
5  venue_team = df.groupby(["venue", "winner"]).size().reset_index(name="wins")
6
7  top = venue_team.loc[venue_team.groupby("venue")["wins"].idxmax()]
8
9  print("Top Team per Venue:\n", top)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Top Team per Venue:
   venue  winner  wins
0  Abu Dhabi  Chennai Super Kings    1
1    Chennai  Chennai Super Kings    1
2    Delhi  Kings XI Punjab    1
3    Dubai  Kings XI Punjab    1
5  Eden Gardens  Kolkata Knight Riders    1
6    Kolkata  Kolkata Knight Riders    1
7    Mumbai  Mumbai Indians    1
8    Sharjah  Delhi Capitals    1
9  Wankhede Stadium  Chennai Super Kings    1
PS C:\Users\mahesh\OneDrive\Desktop\eds activity>
```

PROBLEM STATEMENT5

Find matches where runs are greater than 170.

Input

Column: runs

Output

High scoring matches

```
problem5.py > ...
1  import pandas as pd
2
3  df = pd.read_csv("ipl.csv")
4
5  high = df[df["runs"] > 170]
6
7  print("High Scoring Matches:\n")
8  print(high[["team1", "team2", "runs", "winner"]])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
lem5.py"
High Scoring Matches:

   team1                team2  runs  winner
1  Royal Challengers Bangalore  Kolkata Knight Riders   176  Kolkata Knight Riders
2                Delhi Capitals    Kings XI Punjab   180    Kings XI Punjab
5    Kolkata Knight Riders    Kings XI Punjab   190  Kolkata Knight Riders
6    Mumbai Indians  Kolkata Knight Riders   200    Mumbai Indians
7    Delhi Capitals    Chennai Super Kings   185    Delhi Capitals
8  Royal Challengers Bangalore    Kings XI Punjab   210    Kings XI Punjab
9    Chennai Super Kings  Kolkata Knight Riders   192    Chennai Super Kings
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> █
```

PROBLEM STATEMENT6

Analyze average runs per season.

Input

Columns: season, runs

Output

Average runs per season

```
problem6.py > ...
1  import pandas as pd
2
3  df = pd.read_csv("ipl.csv")
4
5  avg_runs = df.groupby("season")["runs"].mean()
6
7  print("Average Runs per Season:\n", avg_runs)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS +

```
9          Chennai Super Kings  Kolkata Knight Riders  192  Chennai Super Kings
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> & C:/Users/mahesh/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/mahesh/OneDrive/Desktop/eds activity/problem6.py"
Average Runs per Season:
   season
2018    173.666667
2019    170.000000
2020    198.333333
2021    192.000000
Name: runs, dtype: float64
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> |
```

PROBLEM STATEMENT7

Calculate win percentage of each team.

Input

Columns: team1, team2, winner

Output

Win percentage

```
problem7.py > ...
1  import pandas as pd
2
3  df = pd.read_csv("ipl.csv")
4
5  matches = pd.concat([df["team1"], df["team2"]]).value_counts()
6
7  wins = df["winner"].value_counts()
8
9  win_rate = (wins / matches) * 100
10
11 print("Team Win Percentage:\n", win_rate)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> & C:/Users/mahesh/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/mahesh/OneDrive/Desktop/eds activity/problem7.py"
```

Team Win Percentage:

Chennai Super Kings	75.000000
Delhi Capitals	33.333333
Kings XI Punjab	66.666667
Kolkata Knight Riders	50.000000
Mumbai Indians	66.666667
Royal Challengers Bangalore	NaN

Name: count, dtype: float64

```
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> █
```

PROBLEM STATEMENT8

Calculate mean, median, standard deviation of runs.

Input

Column: runs

Output

Statistical values

```
problem8.py > ...  
1  import pandas as pd  
2  import numpy as np  
3  
4  df = pd.read_csv("ipl.csv")  
5  
6  runs = df["runs"]  
7  
8  print("Mean:", np.mean(runs))  
9  print("Median:", np.median(runs))  
10 print("Std Dev:", np.std(runs))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> & C:/Users/mahesh/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/mahesh/OneDrive/Desktop/eds activity/problem8.py"
```

```
Mean: 181.8
```

```
Median: 182.5
```

```
Std Dev: 16.666133324799727
```

```
PS C:\Users\mahesh\OneDrive\Desktop\eds activity>
```

PROBLEM STATEMENT9

Compare wins of team1 vs team2.

Input

Columns: team1, team2, winner

Output

Win counts

```
problem9.py > ...
1  import pandas as pd
2
3  df = pd.read_csv("ipl.csv")
4
5  team1_wins = df[df["team1"] == df["winner"]]
6
7  team2_wins = df[df["team2"] == df["winner"]]
8
9  print("Team1 Wins:", len(team1_wins))
10 print("Team2 Wins:", len(team2_wins))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> & C:/Users/mahesh/OneDrive/Desktop/python/pythoncore-3.14-64/python.exe "c:/Users/mahesh/OneDrive/Desktop/problem9.py"
Team1 Wins: 6
Team2 Wins: 4
```

PROBLEM STATEMENT10

Analyze whether batting or fielding first is better.

Input

Column: toss_decision

Output

Win percentage per decision

```
problem10.py > ...
1  import pandas as pd
2
3  df = pd.read_csv("ipl.csv")
4
5  df["win"] = df["toss_winner"] == df["winner"]
6
7  result = df.groupby("toss_decision")["win"].mean() * 100
8
9  print(result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> & C:/Users/mahesh/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/mahesh/OneDrive/Desktop/eds activity/problem10.py"
toss_decision
bat      40.0
field    20.0
Name: win, dtype: float64
PS C:\Users\mahesh\OneDrive\Desktop\eds activity>
```

PROBLEM STATEMENT11

Predict winner based on historical win rate.

Input

Columns: team1, team2, winner

Output

Predicted winner

```
problem11.py > ...
2
3 df = pd.read_csv("ipl.csv")
4
5 matches = pd.concat([df["team1"], df["team2"]]).value_counts()
6
7 wins = df["winner"].value_counts()
8
9 win_rate = (wins / matches)
10
11 def predict(row):
12     t1 = row["team1"]
13     t2 = row["team2"]
14     return t1 if win_rate.get(t1,0) > win_rate.get(t2,0) else t2
15
16 df["predicted"] = df.apply(predict, axis=1)
17
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
problem11.py"
      team1 ... predicted
0      Mumbai Indians ... Chennai Super Kings
1 Royal Challengers Bangalore ... Kolkata Knight Riders
2      Delhi Capitals ... Kings XI Punjab
3      Mumbai Indians ... Mumbai Indians
4      Chennai Super Kings ... Royal Challengers Bangalore
5 Kolkata Knight Riders ... Kings XI Punjab
6      Mumbai Indians ... Mumbai Indians
7      Delhi Capitals ... Chennai Super Kings
8 Royal Challengers Bangalore ... Kings XI Punjab
9      Chennai Super Kings ... Chennai Super Kings
```

PROBLEM STATEMENT12

Find players who have won Player of the Match multiple times.

Input

Column: player_of_match

Output

Players with more than 1 award

```
problem.py > ...
1  import pandas as pd
2
3  df = pd.read_csv("ipl.csv")
4
5  counts = df["player_of_match"].value_counts()
6
7  consistent = counts[counts > 1]
8
9  print("Consistent Players:\n", consistent)
10
11 total_players = len(counts)
12
13 print("Total Players:", total_players)
14
15 print("Repeated Winners Count:", len(consistent))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> & C:/Users/mahesh/AppData/Local/Programs/Python/pythoncore-3.14-64/python.exe "c:/Users/mahesh/OneDrive/Desktop/eds activity/problem.py"
Consistent Players:
Series([], Name: count, dtype: int64)
Total Players: 10
Repeated Winners Count: 0
PS C:\Users\mahesh\OneDrive\Desktop\eds activity>
```

PROBLEM STATEMENT13

Analyze average runs based on toss decision.

Input

Columns: toss_decision, runs

Output

Average runs for bat/field



The screenshot shows the Visual Studio Code interface. On the left, the 'EDS ACTIVITY' sidebar lists files: ipl.csv, problem1.py, problem2.py, problem3.py, problem4.py, problem5.py, problem6.py, problem7.py, problem8.py, problem9.py, problem10.py, problem11.py, and problem13.py. The main editor displays the code for problem13.py:

```
1 import pandas as pd
2
3 df = pd.read_csv("ipl.csv")
4
5 group = df.groupby("toss_decision")["runs"]
6
7 avg_runs = group.mean()
8
9 print("Average Runs by Toss Decision:\n", avg_runs)
10
11 max_runs = group.max()
12
13 print("Max Runs:\n", max_runs)
```

Below the code editor, the 'TERMINAL' tab is active, showing the command to run the script and its output:

```
python/pythoncore-3.14-64/python.exe "c:/Users/maresh/OneDrive/Desktop/eds act
problem13.py"
Average Runs by Toss Decision:
toss_decision
bat      185.0
field    178.6
Name: runs, dtype: float64
Max Runs:
toss_decision
bat       210
field     192
Name: runs, dtype: int64
PS C:\Users\maresh\OneDrive\Desktop\eds activity>
```

PROBLEM STATEMENT14

Find average runs scored at each venue.

◆ Input

Columns: venue, runs

◆ Output

Venue-wise average runs

The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer lists files under 'EDS ACTIVITY', including 'ipl.csv' and several 'problem' files. 'problem14.py' is selected. The code editor shows the following Python code:

```
problem14.py > ...
1  import pandas as pd
2
3  df = pd.read_csv("ipl.csv")
4
5  avg = df.groupby("venue")["runs"].mean()
6
7  print("Average Runs per Venue:\n", avg)
8
9  count = df["venue"].value_counts()
10
11 print("Matches per Venue:\n", count)
```

Below the code editor, there are tabs for 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active, showing the output of the script:

```
Matches per Venue:
venue
Dubai                2
Wankhede Stadium     1
Eden Gardens         1
Mumbai               1
Delhi                1
Chennai              1
Kolkata              1
Sharjah              1
Abu Dhabi            1
Name: count, dtype: int64
PS C:\Users\mahesh\OneDrive\Desktop\eds activity>
```

PROBLEM STATEMENT15

Find which team won most matches in each season.

Input

Columns: season, winner

Output

Top team per season

EDS ACTIVITY

ipl.csv

problem1.py

problem.py

problem2.py

problem3.py

problem4.py

problem5.py

problem6.py

problem7.py

problem8.py

problem9.py

problem10.py

problem11.py

problem13.py

problem14.py

problem15.py

problem15.py > ...

```
1 import pandas as pd
2
3 df = pd.read_csv("ipl.csv")
4
5 group = df.groupby(["season", "winner"]).size().reset_index(name="wins")
6
7 top = group.loc[group.groupby("season")["wins"].idxmax()]
8
9 print("Best Team per Season:\n", top)
10
11 print("All Season Data:\n", group)
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

All Season Data:

	season	winner	wins
0	2018	Chennai Super Kings	1
1	2018	Kings XI Punjab	1
2	2018	Kolkata Knight Riders	1
3	2019	Chennai Super Kings	1
4	2019	Kolkata Knight Riders	1
5	2019	Mumbai Indians	1
6	2020	Delhi Capitals	1
7	2020	Kings XI Punjab	1
8	2020	Mumbai Indians	1
9	2021	Chennai Super Kings	1

PS C:\Users\mahesh\OneDrive\Desktop\eds activity>

> OUTLINE

> TIMELINE

PROBLEM STATEMENT16

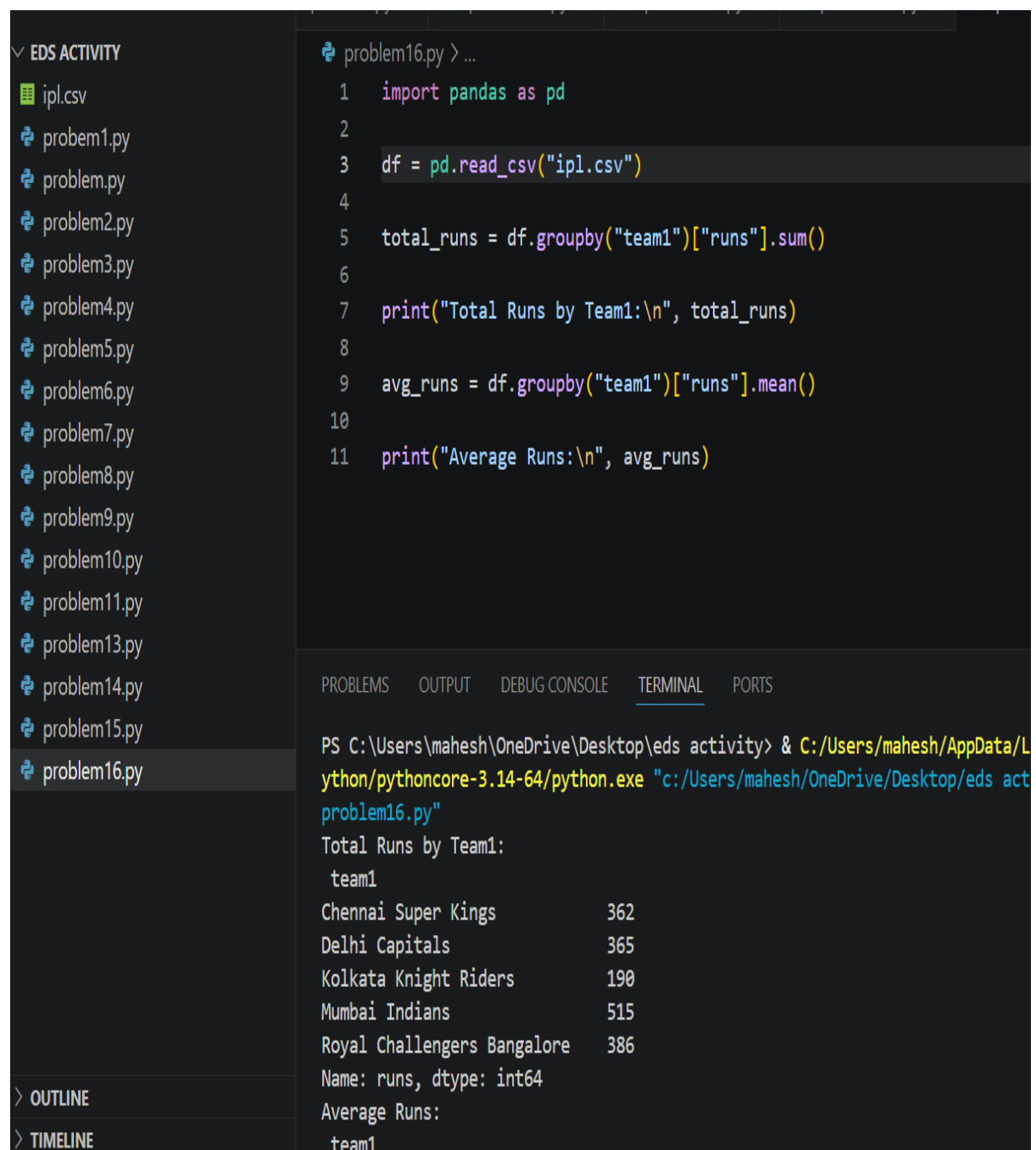
Analyze total runs scored by each team (team1 only).

Input

Columns: team1, runs

Output

Total runs per team



The screenshot shows an IDE with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer lists files like `ipl.csv` and `problem16.py`. The code editor shows a Python script that reads a CSV file, groups data by team, and calculates total and average runs. The terminal shows the command to run the script and its output, which is a table of total runs for each team.

```
problem16.py > ...
1  import pandas as pd
2
3  df = pd.read_csv("ipl.csv")
4
5  total_runs = df.groupby("team1")["runs"].sum()
6
7  print("Total Runs by Team1:\n", total_runs)
8
9  avg_runs = df.groupby("team1")["runs"].mean()
10
11 print("Average Runs:\n", avg_runs)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> & C:/Users/mahesh/AppData/L
ython/pythoncore-3.14-64/python.exe "c:/Users/mahesh/OneDrive/Desktop/eds act
problem16.py"
Total Runs by Team1:
team1
Chennai Super Kings      362
Delhi Capitals            365
Kolkata Knight Riders     190
Mumbai Indians           515
Royal Challengers Bangalore 386
Name: runs, dtype: int64
Average Runs:
team1
```

PROBLEM STATEMENT16

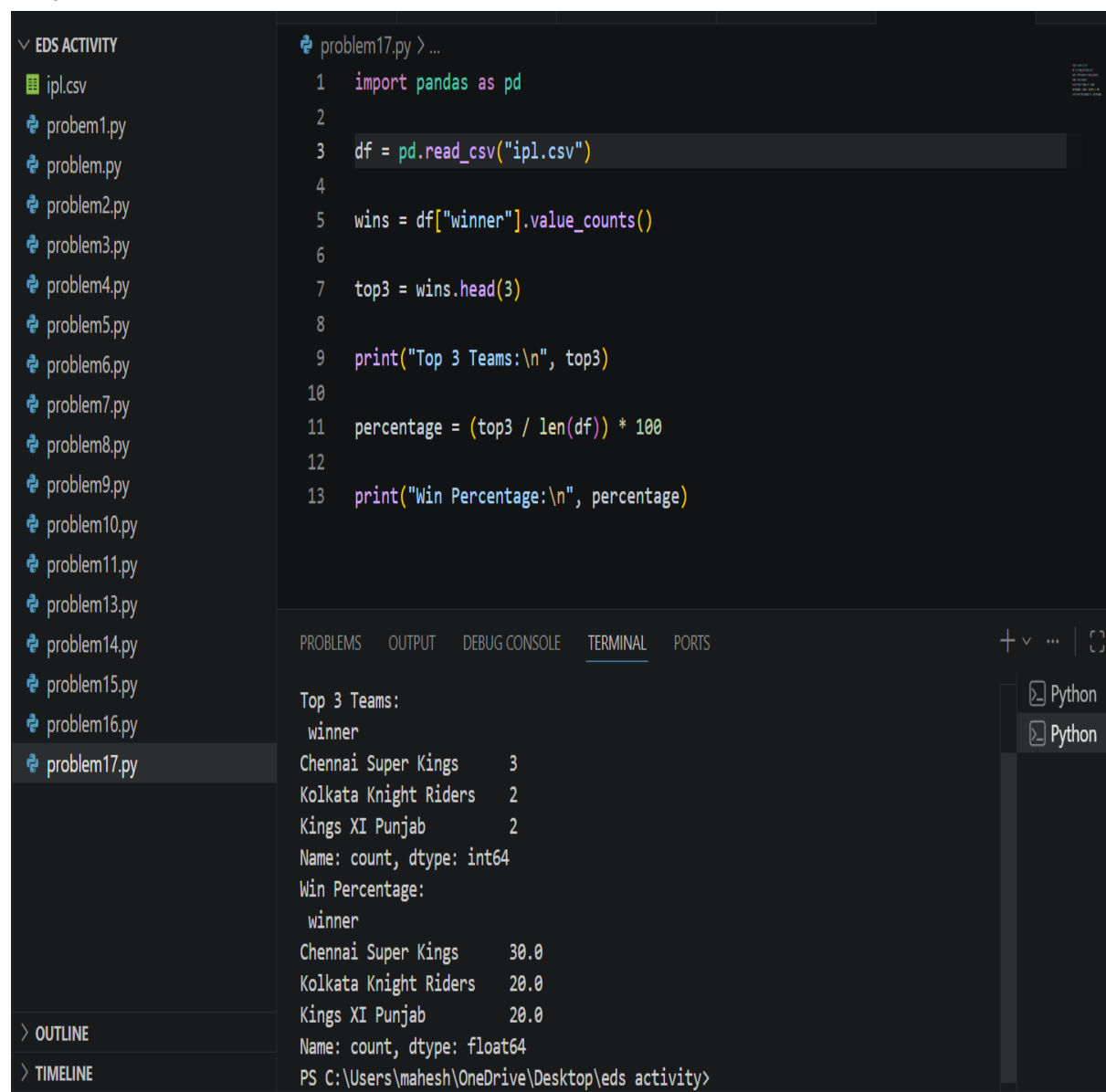
Find top 3 teams based on total wins.

Input

Column: winner

Output

Top 3 teams



The screenshot shows a code editor with a file explorer on the left and a terminal at the bottom. The file explorer lists files from 'ipl.csv' to 'problem17.py'. The code editor displays a Python script in 'problem17.py' that reads 'ipl.csv', counts wins by team, and prints the top 3 teams and their win percentages. The terminal shows the output of the script.

```
problem17.py > ...
1  import pandas as pd
2
3  df = pd.read_csv("ipl.csv")
4
5  wins = df["winner"].value_counts()
6
7  top3 = wins.head(3)
8
9  print("Top 3 Teams:\n", top3)
10
11  percentage = (top3 / len(df)) * 100
12
13  print("Win Percentage:\n", percentage)
```

Terminal Output:

```
Top 3 Teams:
winner
Chennai Super Kings    3
Kolkata Knight Riders  2
Kings XI Punjab        2
Name: count, dtype: int64
Win Percentage:
winner
Chennai Super Kings    30.0
Kolkata Knight Riders  20.0
Kings XI Punjab        20.0
Name: count, dtype: float64
PS C:\Users\mahesh\OneDrive\Desktop\eds activity>
```

PROBLEM STATEMENT18

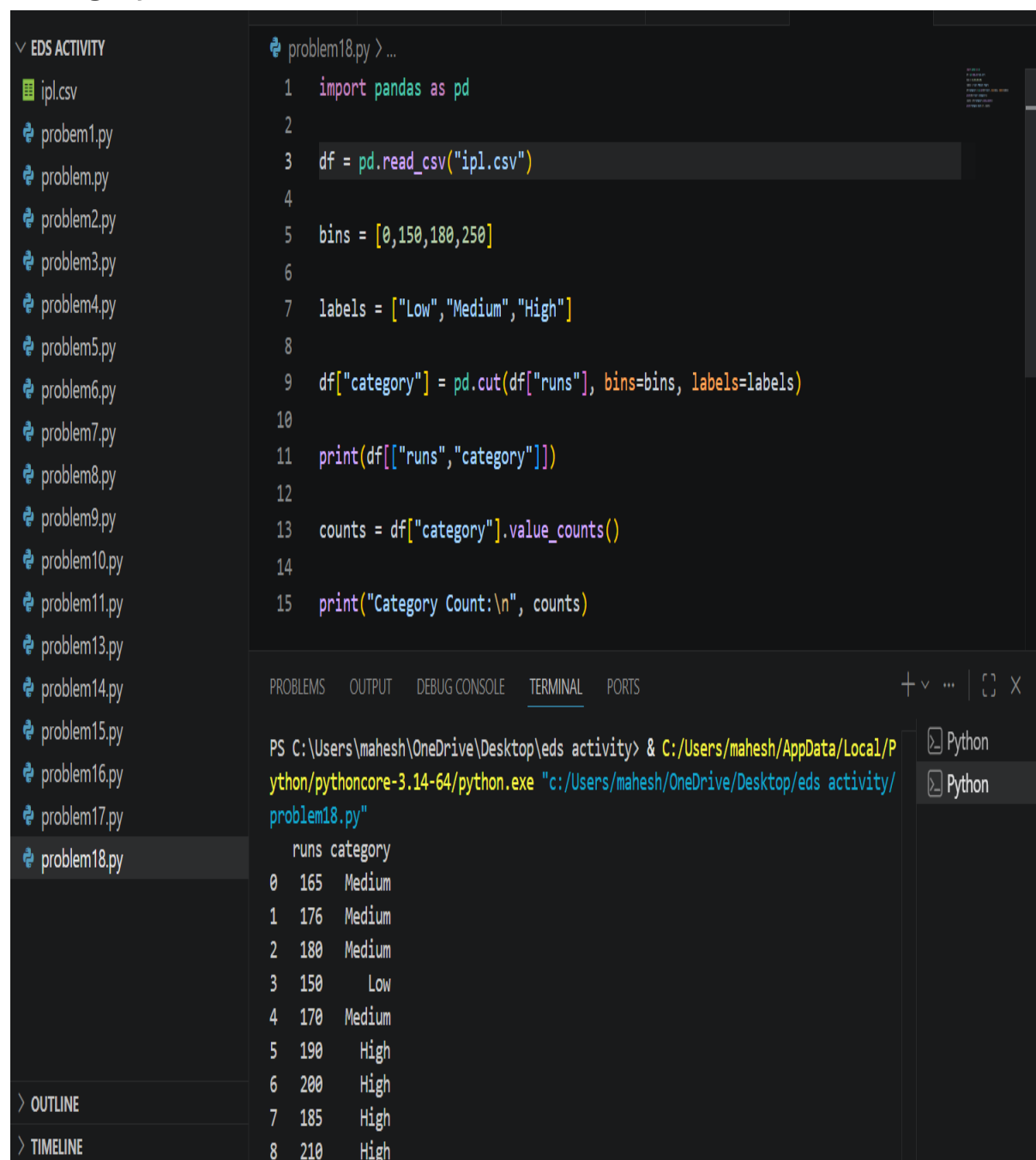
Classify matches into Low, Medium, High scoring.

Input

Column: runs

Output

Category counts



The screenshot shows a code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The file explorer lists files like `ipl.csv` and `problem1.py` through `problem18.py`. The code editor shows the following Python code:

```
1 import pandas as pd
2
3 df = pd.read_csv("ipl.csv")
4
5 bins = [0,150,180,250]
6
7 labels = ["Low", "Medium", "High"]
8
9 df["category"] = pd.cut(df["runs"], bins=bins, labels=labels)
10
11 print(df[["runs", "category"]])
12
13 counts = df["category"].value_counts()
14
15 print("Category Count:\n", counts)
```

The terminal shows the command to run the script and the output:

```
PS C:\Users\mahesh\OneDrive\Desktop\eds activity> & C:/Users/mahesh/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/mahesh/OneDrive/Desktop/eds activity/problem18.py"
runs category
0 165 Medium
1 176 Medium
2 180 Medium
3 150 Low
4 170 Medium
5 190 High
6 200 High
7 185 High
8 210 High
```

PROBLEM STATEMENT19

Analyze wins by runs vs wickets (using wickets column).

Input

Columns: runs, wickets

Output

Count of win types

The screenshot shows a code editor with a file explorer on the left and a terminal at the bottom. The file explorer lists files like `ipl.csv` and `problem1.py` through `problem19.py`. The main editor shows the code for `problem19.py`:

```
1 import pandas as pd
2
3 df = pd.read_csv("ipl.csv")
4
5 df["win_type"] = df["wickets"].apply(lambda x: "Wickets" if x > 5 else "Runs")
6
7 counts = df["win_type"].value_counts()
8
9 print("Win Type Count:\n", counts)
10
11 print(df[["runs", "wickets", "win_type"]])
```

The terminal output shows the result of the `value_counts()` operation:

```
Name: count, dtype: int64
runs  wickets win_type
0    165      5    Runs
1    176      4    Runs
2    180      6  Wickets
3    150      7  Wickets
4    170      5    Runs
5    190      3    Runs
6    200      2    Runs
7    185      4    Runs
8    210      2    Runs
9    192      3    Runs
```

The terminal prompt is `PS C:\Users\mahesh\OneDrive\Desktop\eds activity>`.

PROBLEM STATEMENT20

Find number of matches between each pair of teams.

Input

Columns: team1, team2

Output

Match count per pair

The screenshot shows a code editor with a file explorer on the left and a terminal at the bottom. The file explorer lists several files, with 'problem20.py' selected. The code editor displays a Python script that reads a CSV file 'ipl.csv', groups the data by 'team1' and 'team2', and prints the head-to-head matches and the total number of pairs.

```
problem20.py > ...
1 import pandas as pd
2
3 df = pd.read_csv("ipl.csv")
4
5 pairs = df.groupby(["team1", "team2"]).size().reset_index(name="matches")
6
7 print("Head-to-Head:\n", pairs)
8
9 print("Total Pairs:", len(pairs))
```

The terminal output shows the following table:

	team1	team2	matches
0	Chennai Super Kings	Kolkata Knight Riders	1
1	Chennai Super Kings	Royal Challengers Bangalore	1
2	Delhi Capitals	Chennai Super Kings	1
3	Delhi Capitals	Kings XI Punjab	1
4	Kolkata Knight Riders	Kings XI Punjab	1
5	Mumbai Indians	Chennai Super Kings	1
6	Mumbai Indians	Delhi Capitals	1
7	Mumbai Indians	Kolkata Knight Riders	1
8	Royal Challengers Bangalore	Kings XI Punjab	1
9	Royal Challengers Bangalore	Kolkata Knight Riders	1

Total Pairs: 10

